ADAPTED *VOXELMORPH* DOCUMENTATION

Sarah Liu

DEPENDENCIES:

Code was built using the following package versions:

- Numpy (1.24.3)
- OpenCV (opencv-python==4.10.0.84)
- Pathlib (1.0.1)
- Neurite (0.2)
- SciPy.io (scipy==1.13.1)
- Tqdm (4.66.4)
- PIL (pillow==10.3.0)
- Pytorch (torch==2.3.1+cu118)
- Sklearn (scikit-learn==1.5.0)
- Seaborn (0.13.2)
- Tifffile (2024.5.22)
- Matplotlib (3.9.0)
- VoxelMorph (0.2)
- Tensorflow[and-cuda] (2.18.0, Must use WSL2 to use tensorflow with gpu capabilities)
- Tensorflow-addons (NOT WORKING FOR SOME REASON, comment out lines using tensorflow-addons)

PRE-DEBUGGING PACKAGE UPDATES:

Neurite:

Need to change mean_average_error to MAE, mean_squared_error to MSE and .getargspec(func) to .getfullargspec(func)[:4] in neurite files:

neurite\tf\metrics.py, neurite\tf\losses.py:

```
31  # simple metrics renamed mae -> l1, mse -> l2
32  from tensorflow.keras.losses import MAE as l1
33  from tensorflow.keras.losses import MSE as l2
```

neurite\tf\modelio.py:

```
 8  def store_config_args(func):
 9      """
10      Class-method decorator that saves every argument provided to the
11      function as a dictionary in 'self.config'. This is used to assist
12      model loading - see LoadableModel.
13      """
14
15      attrs, varargs, varkw, defaults = inspect.getfullargspec(func)[:4]
16
```

VoxelMorph:

Change .get_shape() to just .shape in voxelmorph files:

voxelmorph\tf\networks.py

```
1072            raise ValueError( cannot use nb_le
1073
1074        ndims = len(unet_input.shape) - 2
1075        assert ndims in (1, 2, 3), 'ndims shou
```

```
1198      Specific convolutional block followed by leakyrelu for unet.
1199      """
1200      ndims = len(x.shape) - 2
1201      assert ndims in (1, 2, 3), 'ndims should be one of 1, 2, or 3
```

```
1233      Specific upsampling and concatenation layer for unet.
1234      """
1235      ndims = len(x.shape) - 2
1236      assert ndims in (1, 2, 3), 'ndims should be one of 1,
1237      UpSampling = getattr(KL, 'UpSampling%dD' % ndims)
1238
```

voxelmorph\torch\modelio.py

```
def store_config_args(func):
    """
    Class-method decorator that saves every argument provided to the
    function as a dictionary in 'self.config'. This is used to assist
    model loading - see LoadableModel.
    """

    attrs, varargs, varkw, defaults = inspect.getfullargspec(func)[:4]

    @functools.wraps(func)
```

ReduceLROnPlateau(Callback):
Change self.model.optimizer.lr to learning_rate instead

```
def on_epoch_end(self, epoch, logs=None):
  logs = logs or {}
  logs['lr'] = backend.get_value(self.model.optimizer.learning_rate)
  current = logs.get(self.monitor)
  if current is None:
    logging.warning('Learning rate reduction is conditioned on metric `%s` '
                    'which is not available. Available metrics are: %s',
                    self.monitor, ','.join(list(logs.keys())))

  else:
    if self.in_cooldown():
      self.cooldown_counter -= 1
      self.wait = 0

    if self.monitor_op(current, self.best):
      self.best = current
      self.wait = 0
    elif not self.in_cooldown():
      self.wait += 1
      if self.wait >= self.patience:
        old_lr = backend.get_value(self.model.optimizer.learning_rate)
        if old_lr > np.float32(self.min_lr):
          new_lr = old_lr * self.factor
          new_lr = max(new_lr, self.min_lr)
          backend.set_value(self.model.optimizer.learning_rate, new_lr)
          if self.verbose > 0:
            print('\nEpoch %05d: ReduceLROnPlateau reducing learning '
                  'rate to %s.' % (epoch + 1, new_lr))
          self.cooldown_counter = self.cooldown
          self.wait = 0
```

FILE DESCRIPTIONS:
There are two types of executable files: ***train*_{suffix}*.py*** and ***test*_{suffix}*.py***. Use 'debug' parameter to enable faster execution of the whole script (shortened data loading and training).

NOTES
Using vs code, pip install necessary packages on 3.7.9 or newer
Download the CAMUS dataset from
https://humanheart-project.creatis.insa-lyon.fr/database/#collection/6373703d73e9f0047faa1bc8

DIRECTIONS:

1. Process data (CAMUS and simulation) using MATLAB scripts (existing processed data on "Sarah Summer 2024" drive)
    a. Matlab scripts also on drive
2. Run the train file corresponding to the type of data
    a. Ex. for CAMUS data, run train_CAMUS
3. After training, run corresponding tes file
    a. Ex. for CAMUS data, run test_CAMUS
4. Main is a visualization tool, after test can see results

train*_{suffix}*.py files are run to create and train a new instance of VoxelMorph:

- Import statements
- Model hyperparameters: all file paths and training parameters should be edited here
- Data preprocessing: the data is organized into 'moving' (source) and 'fixed' (target) numpy array datasets and then fed into custom generators
    - Dataset processing is currently customized to different file setups using specific folder names/structures; may need to edit to fit specific data
- Model creation: the model is defined here, model layers, optimizer and loss functions can be edited here
- Training: Loops through epochs; each epoch has a training and validation stage
    - Calculates simulation loss ($\mathcal{L}_{sim}$: penalizes differences between predicted and real) and smoothing loss ($\mathcal{L}_{smooth}$: penalizes spatial differences in deformation field ϕ) for each batch, adds losses together to represent loss for entire batch, which is backpropagated through model
        - Losses can be changed/defined in the ***losses.py*** file
    - Average loss across all batches represents loss for the epoch
    - Following training stage, validation stage calculates loss for each validation batch and averages to find validation loss for the epoch
        - Validation loss for each epoch is recorded and saved to 'val_losses.txt'
    - If epoch validation loss is less than previous, save new model weights
        - New weights are saved to 'model_weights.pth'

test*_{suffix}*.py files are used to benchmark the model and follow a similar structure to the training files:

- Import statements
- Model hyperparameters
- Data preprocessing
- Model loading: the model is loaded here from a previously-specified weights file
- Prediction: Loops through testing generator, predicts deformation field ϕ and generates "moved" image (source or "moving" image convolved with ϕ, targeted to be similar to consecutive or "fixed" image)
    - Predicts flow for each image pair, then sums all flow values to find cumulative flow for entire sequence
- Visualization: includes several plots to visualize deformation field and displacement estimation

generators.py and ***losses.py*** are Voxelmorph scripts that have been changed by implementing custom losses/generator objects. All the executable files use generators.custom_generator (one type of data) or generators.generator_4D (joint rf and b mode) to feed image pairs into the model.

bmode_rf_network.py has all the code for defining and creating the joint model, and is written in keras/tensorflow. All executable files with the suffix ***_keras*** call the file's function, Vxm4D, to help build the model.

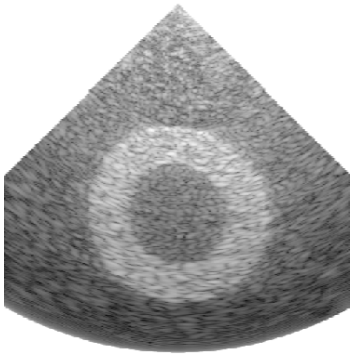
IUS_Accuracy.py is a script I wrote to get the average axial and lateral displacement from predicted outputs from the joint model.

main.py is for visualizing a series of true/predicted images next to each other using a custom class SliceViewer. Right now they are set up to show one whole circular systole simulation, but can be altered to show individual before/after predictions.

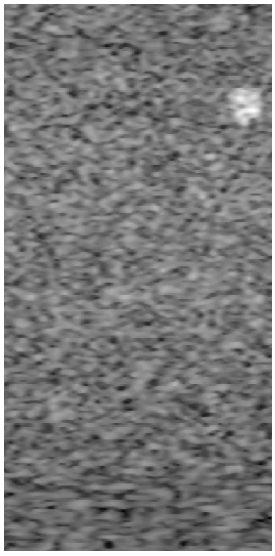
I don't remember what has to happen with the ***__init__.py*** and ***setup.py*** files, but I think they have to be in the directory for the code to work. (Still not sure but was able to run somewhat without them)

{suffix} KEY AND DATA FORMATTING:

- ***_jad*** files are VoxelMorph training/testing files for the circular systole heart simulation
    - One training file is built to process B-modes, other for the RF data
    - Data format: Data is organized into five overarching folders. Within these general folders, individual folders each hold the .mat data for single sequences, including B-mode data, RF data and the simulation masks. One sequence pictures the simulation from the beginning to end of 'systole.'

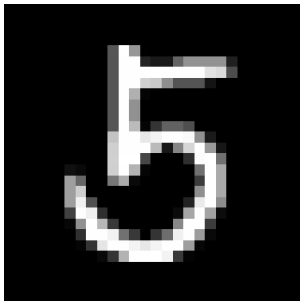


- ***_sim*** files are training/testing files for the moving spot simulation B-modes
    - Image frames are cropped so that only the simulation appears; excludes matlab axes and border
    - Data format: Vertical and horizontal sequences are organized into folders labeled by the movement direction and coordinates. Within folders, ordered .png images are labeled with coordinates of the pictured inclusion.



- ***_CAMUS*** files are for the CAMUS dataset
    - 'img_test_2CH_contrast': Images are contrast-enhanced using ImageJ and cropped to 512 x 512 pixels
    - Data format: Ultrasound scans are organized into a training/validation folder and a testing folder. Each scan is a 3-dimensional .tif file, where the heart is pictured from the beginning to end of systole.

- ***_mnist*** files are tester files built to verify that VoxelMorph is working correctly
    - Downloads MNIST dataset directly from sklearn databases
    - Data format: Data is downloaded from keras online databases.



    - Debugging for MNIST dataset download (“fetch_openml”): navigate to Python version in file explorer, run “Install Certificates.command” file. (May have to run twice.)
- ***_terason*** files are tester files for the terason data. Right now, these files only take individual acquisitions and train on the same image pairs many times, but they can be easily edited to take larger groups of acquisitions in a similar way to other scripts. ***train_terason_bmode_rf_keras.py*** has the majority of my comments, which can also be used to help navigate other scripts as well.
- ***_greg_rf*** file is for applying VoxelMorph to Greg’s mystery data
- ***_keras.py*** files are the most up-to-date and most recently written. All the model handling is done through keras through tensorflow.
- ***_bmode_rf*** files use both the b-mode and the radiofrequency data jointly. These files reference ***bmode_rf_network.py*** to construct the joint model.
- ***_concat*** files are scripts I briefly wrote to try to brute force concatenate RF data to B-mode images. I don’t think they work, but maybe they could be adapted to work eventually.